

Here's an example of the traceback that Python provides after you've accidentally misspelled a variable's name:

```
Traceback (most recent call last):
❶ File "hello_world.py", line 2, in <module>
❷   print(message)
      ^^^^^
❸ NameError: name 'message' is not defined. Did you mean: 'message'?
```

The output reports that an error occurs in line 2 of the file *hello_world.py* ❶. The interpreter shows this line ❷ to help us spot the error quickly and tells us what kind of error it found ❸. In this case it found a *name error* and reports that the variable being printed, *message*, has not been defined. Python can't identify the variable name provided. A name error usually means we either forgot to set a variable's value before using it, or we made a spelling mistake when entering the variable's name. If Python finds a variable name that's similar to the one it doesn't recognize, it will ask if that's the name you meant to use.

In this example we omitted the letter *s* in the variable name *message* in the second line. The Python interpreter doesn't spellcheck your code, but it does ensure that variable names are spelled consistently. For example, watch what happens when we spell *message* incorrectly in the line that defines the variable:

```
message = "Hello Python Crash Course reader!"
print(message)
```

In this case, the program runs successfully!

```
Hello Python Crash Course reader!
```

The variable names match, so Python sees no issue. Programming languages are strict, but they disregard good and bad spelling. As a result, you don't need to consider English spelling and grammar rules when you're trying to create variable names and writing code.

Many programming errors are simple, single-character typos in one line of a program. If you find yourself spending a long time searching for one of these errors, know that you're in good company. Many experienced and talented programmers spend hours hunting down these kinds of tiny errors. Try to laugh about it and move on, knowing it will happen frequently throughout your programming life.

Variables Are Labels

Variables are often described as boxes you can store values in. This idea can be helpful the first few times you use a variable, but it isn't an accurate way to describe how variables are represented internally in Python. It's much better to think of variables as labels that you can assign to values. You can also say that a variable references a certain value.